Policy Gradient Algorithms

(basis of REINFORCE, A2C, TRPO, PPO, others)

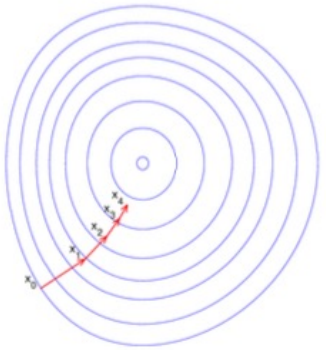
- Let $J(\theta)$ be any policy objective function
- Policy gradient algorithms search for a *local* maximum in $J(\theta)$ by ascending the gradient of the policy, w.r.t. parameters θ

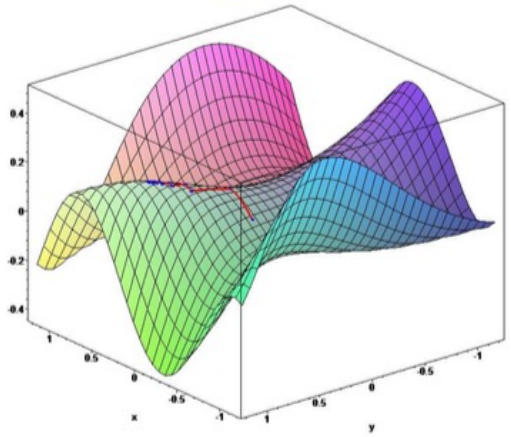
$$\Delta\theta = \alpha \nabla_\theta J(\theta)$$

- Where $\nabla_\theta J(\theta)$ is the policy gradient

$$\nabla_\theta J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

- and α is a step-size parameter





[Abbeel]

Computing the Policy Gradient via finite-differences

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon}$$

- very slow
  - one gradient computation requires n+1 policy evaluations for n policy parameters
- better idea: get a (noisy) policy gradient for each policy time step!

Policy Gradient RL: Intuition

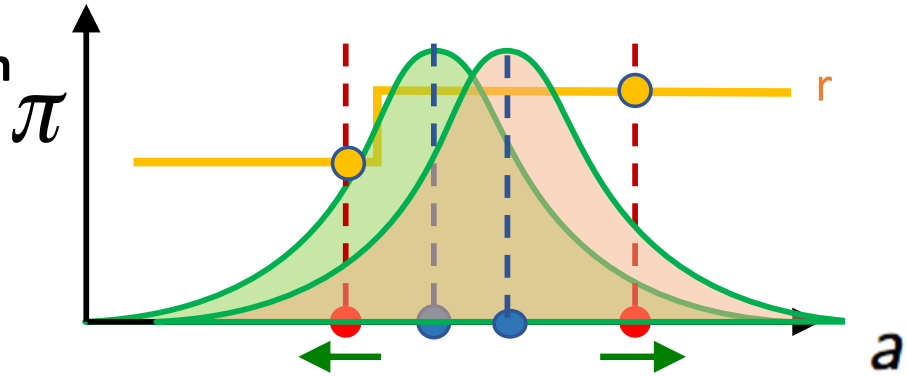
Consider a single-action, one-step reward problem

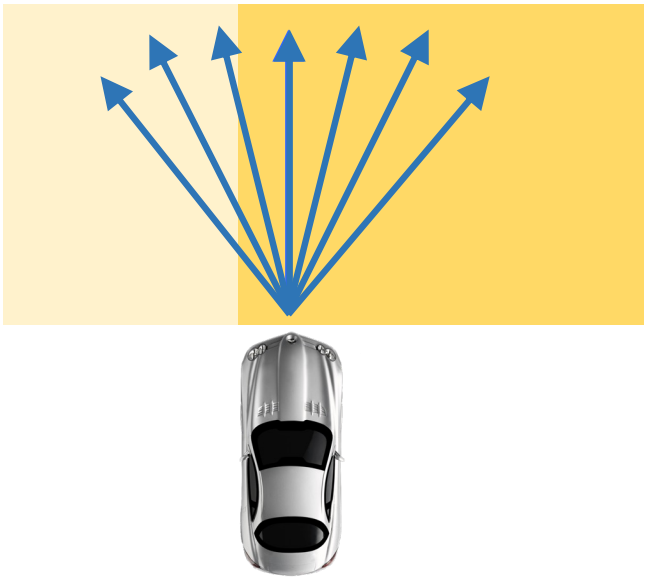
Assume a stochastic policy: $a \sim \mathcal{N}(\mu, \sigma^2)$

where $\mu = f_\theta(s)$ using a neural net (NN)

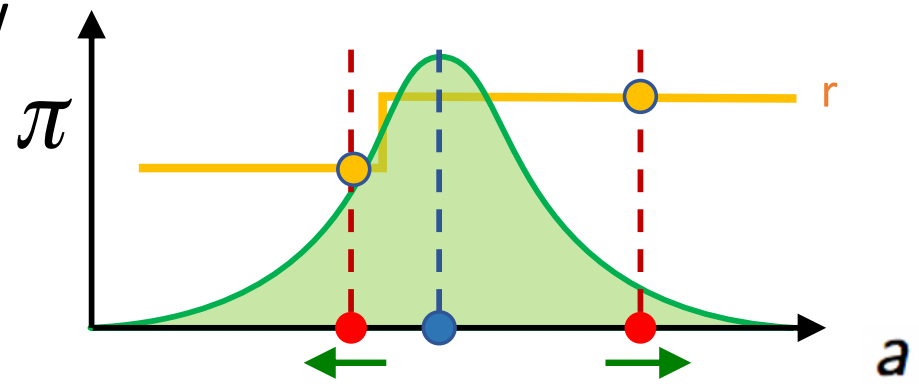
$$J = \int_a \pi(a) R(a)\, \mathrm{d}a = \mathbb{E}_\pi[R]$$

"make the sampled rewards more likely"
(by shifting the mean)





Policy Gradient -- preview



$$\nabla_\mu J = \mathbb{E}_\pi[\frac{\nabla_\mu \pi(a)}{\pi(a)} R]$$

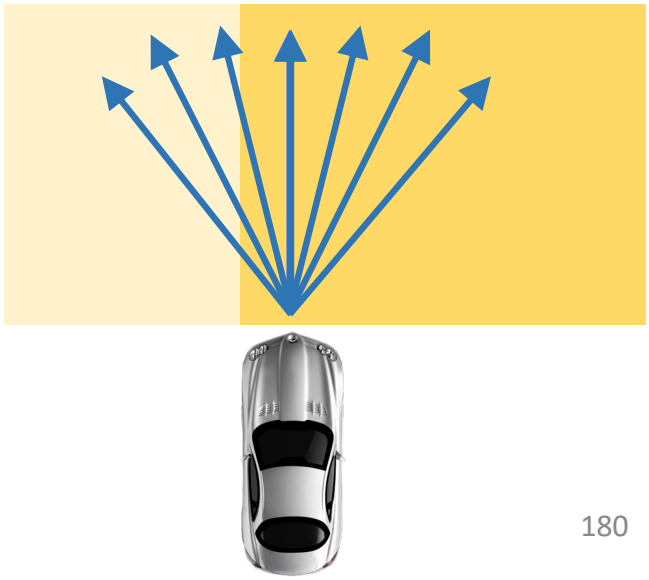
$$\nabla_\mu J = \mathbb{E}_\pi[\nabla_\mu \log \pi(a) R]$$

gradient vectors (score function)

reward

$$\nabla_\theta J = \mathbb{E}_\pi[\nabla_\theta \log \pi(a) R]$$

gradient wrt NN



$$\theta \leftarrow \theta + \alpha \nabla_\theta J$$ take a step uphill

Proof of the Policy Gradient Theorem (episodic case)

With just elementary calculus and re-arranging of terms, we can prove the policy gradient theorem from first principles. To keep the notation simple, we leave it implicit in all cases that π is a function of $\boldsymbol{\theta}$, and all gradients are also implicitly with respect to $\boldsymbol{\theta}$. First note that the gradient of the state-value function can be written in terms of the action-value function as

$$
\begin{aligned}
\nabla v_\pi(s) &= \nabla \left[\sum_a \pi(a|s) q_\pi(s,a) \right], \quad \text{for all } s \in \mathcal{S} && \text{(Exercise 3.18)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla q_\pi(s,a) \Big] && \text{(product rule of calculus)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla \sum_{s',r} p(s',r|s,a) \big(r + v_\pi(s')\big) \Big] \\
& && \text{(Exercise 3.19 and Equation 3.2)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) \nabla v_\pi(s') \Big] && \text{(Eq. 3.4)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) && \text{(unrolling)} \\
&\qquad \sum_{a'} \big[\nabla \pi(a'|s') q_\pi(s',a') + \pi(a'|s') \sum_{s''} p(s''|s',a') \nabla v_\pi(s'') \big] \Big] \\
&= \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \to x, k, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a),
\end{aligned}
$$

after repeated unrolling, where $\Pr(s \to x, k, \pi)$ is the probability of transitioning from state s to state x in k steps under policy π. It is then immediate that

$$
\begin{aligned}
\nabla J(\boldsymbol{\theta}) &= \nabla v_\pi(s_0) \\
&= \sum_s \left(\sum_{k=0}^{\infty} \Pr(s_0 \to s, k, \pi) \right) \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_s \eta(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(box page 199)} \\
&= \sum_{s'} \eta(s') \sum_s \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_{s'} \eta(s') \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Eq. 9.3)} \\
&\propto \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Q.E.D.)}
\end{aligned}
$$

p.325 [Sutton & Barto, 2018]

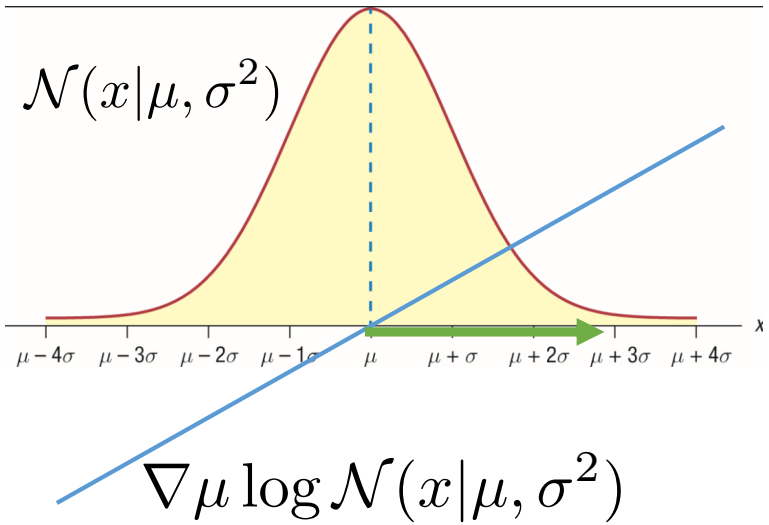
Policy Gradient math (1)

Policy Gradient – math (2)

Policy Gradient math (3)

Gradient wrt mean of log Normal distribution

$$N(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$



$$\nabla_\mu \ln(N(x)) = \nabla_\mu \ln(\frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}})$$

$$= \nabla_\mu(-\frac{1}{2}\ln(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2})$$

$$= \frac{1}{\sigma^2}(x-\mu)$$

$\nabla\mu \log \mathcal{N}(x|\mu, \sigma^2)$

or

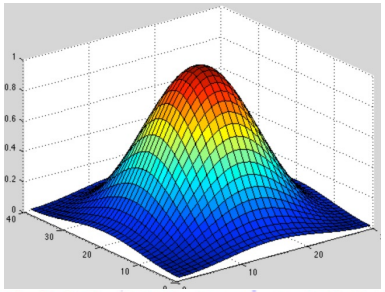
Policy Gradient math (4)

$$\nabla_\mu J \approx \frac{1}{m}\sum_{i=1}^{m} \nabla_\mu \log \pi(a_i|s) R_i$$

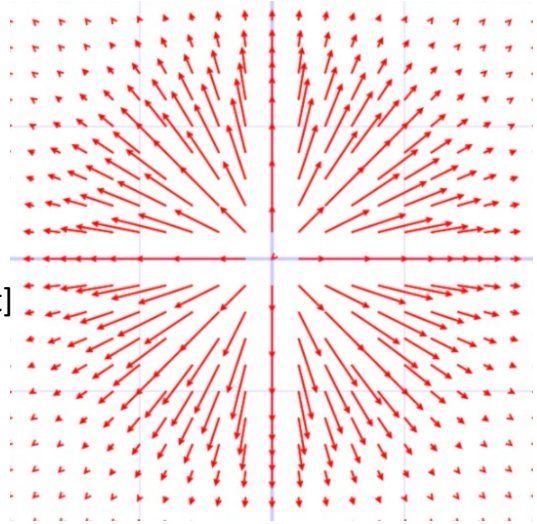
$$\nabla_\mu J \approx \frac{1}{m}\sum_{i=1}^{m} \frac{1}{\sigma^2}(a_i - \mu) R_i$$

2D Stochastic Action Space
with Normal distribution

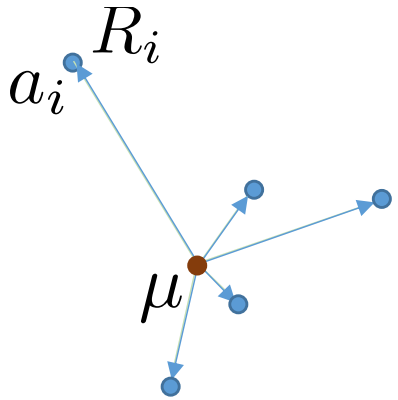
$\pi(a|s)$



$\nabla_\mu \pi(a|s)$



[diagram illustrates the negative gradient]



$$\frac{\nabla_\mu \pi(a|s)}{\pi(a|s)} = \nabla_\mu \log \pi(a|s) = \frac{1}{\sigma^2}(a - \mu)$$

Policy Gradient math (5)

Motivation for introducing a baseline

$$\nabla_\mu J \approx \frac{1}{m}\sum_{i=1}^{m} \nabla_\mu \log \pi(a_i|s) r_i$$

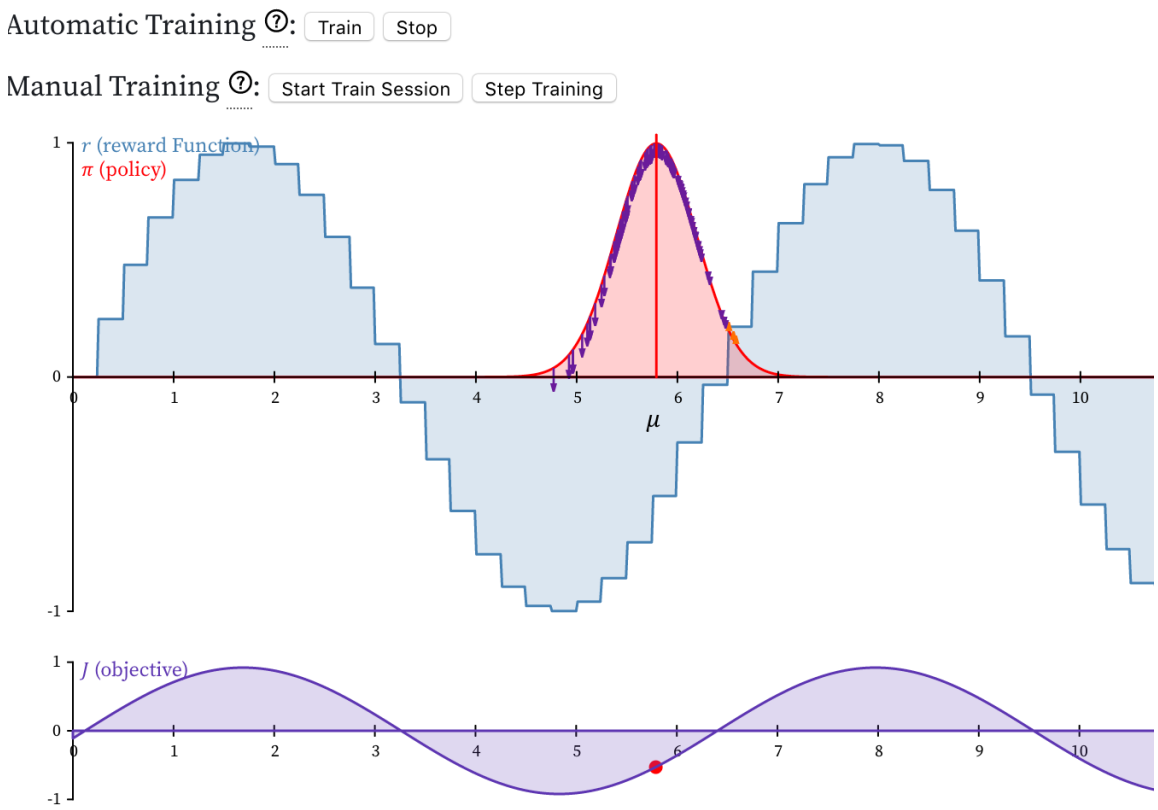
$$\nabla_\mu J \approx \frac{1}{m}\sum_{i=1}^{m} \frac{1}{\sigma^2}(a_i - \mu) r_i$$

Policy Gradient math (6)

Does introducing a baseline change the gradient estimate?

Policy Gradient Demos

[Farzad Abdolhosseini]
https://observablehq.com/@farzadab/policy-gradients



Comments on Policy Gradient methods

- on-policy method: always sample from the current policy
- role of noisy policy
  - needed to compute the gradient
  - exploration
  - policy robustness
  - caveat: alters the original problem
- can also optimize the variance!
  - caveat: often shrinks prematurely
- usually: add noise in action space
  Also valid: add noise directly to policy parameters (NN weights)

$$\text{maximize}_u \quad R(u) \qquad \text{vs}$$
$$\text{maximize}_{p(u)} \quad \mathbb{E}_p[R(u)]$$

["Parameter Exploring Policy Gradients", 2010]

Policy Gradient – Caveats? (sample inefficient)

Our first generic candidate for solving reinforcement learning is *Policy Gradient*. I find it shocking that Policy Gradient wasn't ruled out as a bad idea in 1993. Policy gradient is seductive as it apparently lets one fine tune a program to solve any problem without any domain knowledge. Of course, anything that makes such a claim must be too general for its own good. Indeed, if you dive into it, **policy gradient is nothing more than random search dressed up in mathematical symbols and lingo**.

Lots of papers have been applying policy gradient to all sorts of different settings, and claiming crazy results, but I hope that it is now clear that they are just dressing up random search in a clever outfit. When you end up with a bunch of papers showing that genetic algorithms are competitive with your methods, this does not mean that we've made an advance in genetic algorithms. It is far more likely that this means that your method is a lousy implementation of random search.

[An Outsider's Tour of Reinforcement Learning
http://www.argmin.net/2018/06/25/outsider-rl/ – Ben Recht]

Policy Gradient Derivation for multi-step trajectories

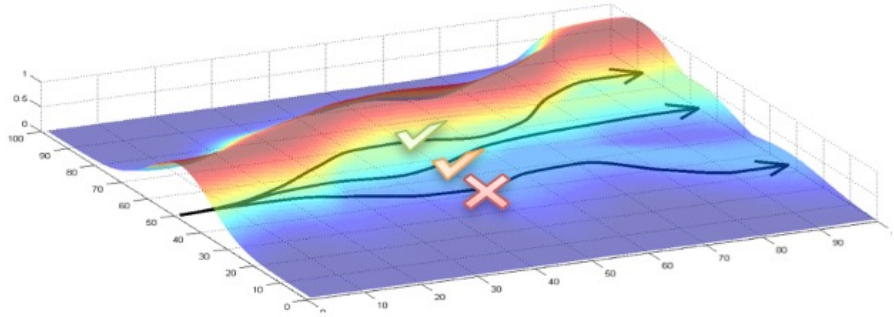
[Abbeel]

We let τ denote a state-action sequence $s_0, u_0, \ldots, s_H, u_H$. We overload notation: $R(\tau) = \sum_{t=0}^{H} R(s_t, u_t)$.

$$U(\theta) = \mathrm{E}[\sum_{t=0}^{H} R(s_t, u_t); \pi_\theta] = \sum_{\tau} P(\tau; \theta) R(\tau)$$

In our new notation, our goal is to find θ:

$$\max_\theta U(\theta) = \max_\theta \sum P(\tau; \theta) R(\tau)$$



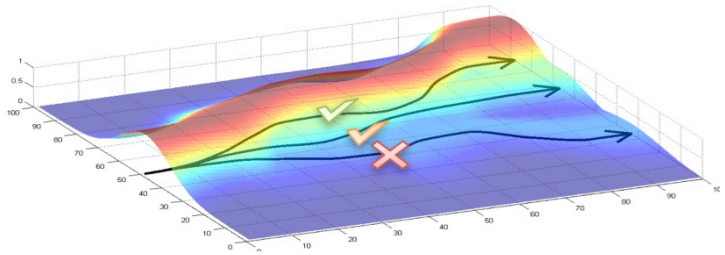
analogous to what we had:

$$\nabla_\mu J \approx \frac{1}{m}\sum_{i=1}^{m} \nabla_\mu \log \pi(a_i|s) r_i$$

[Abbeel]

$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)};\theta) R(\tau^{(i)})$$

- Gradient tries to:
  - Increase probability of paths with positive R
  - Decrease probability of paths with negative R



! Likelihood ratio changes probabilities of experienced paths, does not try to change the paths (<-> Path Derivative)

From paths to state sequences

[Abbeel]

$$\nabla_\theta \log P(\tau^{(i)};\theta) = \nabla_\theta \log \left[\prod_{t=0}^{H} \underbrace{P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{policy}} \right]$$

$$= \nabla_\theta \left[\sum_{t=0}^{H} \log P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)}) + \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)}) \right]$$

$$= \nabla_\theta \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)})$$

$$= \sum_{t=0}^{H} \underbrace{\nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{no dynamics model required!!}}$$

[Abbeel]

The following expression provides us with an unbiased estimate of the gradient, and we can compute it without access to a dynamics model:

$$\hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

Here:

$$\nabla_\theta \log P(\tau^{(i)}; \theta) = \sum_{t=0}^{H} \underbrace{\nabla_\theta \log \pi_\theta(u_t^{(i)} | s_t^{(i)})}_{\text{no dynamics model required!!}}$$

Unbiased means:

$$\mathrm{E}[\hat{g}] = \nabla_\theta U(\theta)$$

REINFORCE

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

[Sutton Barto, p328]

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_\theta \mathbb{E}\left[\sum_{t=0}^{\infty} r_t\right]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E}\left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right], \tag{1}$$

Many choices of "compatible" rewards

where Ψ_t may be one of the following:

1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t.
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.

The latter formulas use the definitions

$$V^\pi(s_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \qquad Q^\pi(s_t, a_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t+1:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \tag{2}$$

$$A^\pi(s_t, a_t) := Q^\pi(s_t, a_t) - V^\pi(s_t), \quad \text{(Advantage function)}. \tag{3}$$

[Schulman 2016]

Generalized Advantage Estimation (GAE)

$$
\begin{aligned}
\hat{A}_t^{(1)} &:= \delta_t^V &&= -V(s_t) + r_t + \gamma V(s_{t+1}) \\
\hat{A}_t^{(2)} &:= \delta_t^V + \gamma\delta_{t+1}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \\
\hat{A}_t^{(3)} &:= \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})
\end{aligned}
$$

$$
\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})
$$

[Schulman 2016]

$$
\begin{aligned}
\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} &:= (1-\lambda)\left(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots\right) \\
&= (1-\lambda)\left(\delta_t^V + \lambda(\delta_t^V + \gamma\delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V) + \dots\right) \\
&= (1-\lambda)(\delta_t^V(1 + \lambda + \lambda^2 + \dots) + \gamma\delta_{t+1}^V(\lambda + \lambda^2 + \lambda^3 + \dots) \\
&\qquad + \gamma^2\delta_{t+2}^V(\lambda^2 + \lambda^3 + \lambda^4 + \dots) + \dots) \\
&= (1-\lambda)\left(\delta_t^V\left(\frac{1}{1-\lambda}\right) + \gamma\delta_{t+1}^V\left(\frac{\lambda}{1-\lambda}\right) + \gamma^2\delta_{t+2}^V\left(\frac{\lambda^2}{1-\lambda}\right) + \dots\right) \\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V
\end{aligned}
\tag{16}
$$

The construction we used above is closely analogous to the one used to define TD(λ) (Sutton & Barto, 1998), however TD(λ) is an estimator of the value function, whereas here we are estimating the advantage function.

[Schulman 2016]

Actor-Critic Policy Gradient Algorithm

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

[Sutton Barto, p333]

In Practice: batching

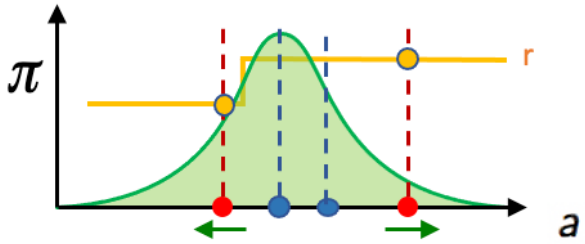
A proximal policy optimization (PPO) algorithm that uses fixed-length trajectory segments is shown below. Each iteration, each of N (parallel) actors collect T timesteps of data. Then we construct the surrogate loss on these NT timesteps of data, and optimize it with minibatch SGD (or usually for better performance, Adam [KB14]), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

[PPO paper: Schulman 2017]

Policy Gradients – review



- stochastic policy
- one-step example

$$J = \int_a \pi(a) R(a)\, \mathrm{d}a = \mathbb{E}_\pi[R]$$

$$\nabla_\mu J = \mathbb{E}_\pi[\frac{\nabla_\mu \pi(a)}{\pi(a)} R]$$

$$\nabla_\mu J = \mathbb{E}_\pi[\nabla_\mu \log \pi(a) R]$$

$$\nabla_\theta J = \mathbb{E}_\pi[\nabla_\theta \log \pi(a) R]$$

- this generalizes to trajectories:

$$\nabla_\theta J \approx \hat{g} = \mathbb{E}_t[\nabla_\theta \log P(\tau;\theta) R(\tau)]$$

- ... and to individual state transitions:

$$\hat{g} = \hat{\mathbb{E}}_t\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\right]$$

with any of: total rewards, future rewards, baseline-adjusted future rewards, TD residual, Q(s,a), Advantage; this yields any of: REINFORCE, "vanilla policy gradient", or A2C algorithm, depending on the choices above)

- given gradient, how to control step size of the update?